

NeuronLab: BCI framework for the study of biosignals

Sergio López Bernal ^{a,*}, Juan Antonio Martínez López ^a, Enrique Tomás Martínez Beltrán ^a,
Mario Quiles Pérez ^a, Gregorio Martínez Pérez ^a, Alberto Huertas Celdrán ^b

^a Department of Information and Communications Engineering, University of Murcia, Murcia, 30100, Spain

^b Communication Systems Group CSG, Department of Informatics IfI, University of Zurich UZH, CH—8050 Zürich, Switzerland

ARTICLE INFO

Communicated by K. Liu

Keywords:

Brain–Computer Interfaces
Electroencephalography
Framework
Frontend
Backend

ABSTRACT

Brain–Computer Interfaces (BCIs) allow the acquisition of brain activity using non-invasive techniques such as Electroencephalography (EEG). Since BCI devices do not commonly interpret the acquired EEG signals, external software applications play a critical role in the BCI lifecycle. Despite the literature offering a great variety of platforms and frameworks, they present several limitations, such as implementing old software architectures or needing more functionality to cover all phases of the BCI lifecycle. Based on these limitations, this work proposes the design and implementation of NeuronLab, a secure, multi-platform, standalone, multi-paradigm, and web-based framework that defines all BCI lifecycle phases and provides novel functionality compared to current open-source BCI software, such as sharing experiments between researchers and storing data on the cloud. This framework has been validated in two experiments common in BCI literature: P300 identification and limb movements detection. This verification has been performed based on performance metrics, such as CPU and RAM consumption, highlighting that NeuronLab is a promising solution for BCI scenarios requiring a distributed and collaborative platform for researchers and practitioners.

1. Introduction

Biosensors emerged in the 1960s to diagnose and prevent diseases, allowing to acquire heterogeneous information from the organism, such as chemical, electrical or optical signals [1]. Nowadays, research concerning biosensors is becoming one of the most studied disciplines contributing to the next generation of medical advances, such as individualized medicine and ultra-sensitive disease detection [2]. In particular, the advances in biosensors have enormously benefited the neuroscience field, allowing the study of the brain and, specifically, identifying the state of mental processes by measuring brain activity [3].

In this context, Brain–Computer Interfaces (BCIs) can be defined as interfaces able to establish bidirectional communication between a subject's brain and external devices. On the one hand, BCIs can stimulate or inhibit neural activity, widely used to treat neurodegenerative diseases such as Parkinson's [4]. On the other hand, these interfaces can acquire neural activity for various purposes, such as identifying brain conditions or controlling external devices mentally. For neural data acquisition, non-invasive BCI devices are typically used due to their reduced price and risk, and high portability. In particular, Electroencephalography (EEG) is the most common BCI technique as it

allows to acquire neurological data with a high temporal resolution by placing non-invasive electrodes on the scalp [5,6].

All the processes required, from acquiring neural signals to determining the action to perform over external devices, are defined in the BCI lifecycle, composed of several phases. Since BCIs have constrained computational capabilities, most of these stages are typically performed by external BCI software. In particular, the BCI lifecycle is usually composed of the following phases [7]: (i) data acquisition, where the BCI device captures the neural data; (ii) preprocessing, where the acquired neural data are filtered to remove components or artifacts that negatively affect the signal; (iii) feature extraction, which identifies relevant characteristics existing in brain signals, reducing the complexity for the next stage; and (iv) classification, whose objective is to identify the action mentally intended by the subject, typically using artificial intelligence techniques.

Although there is a wide variety of open-source BCI software platforms in the literature, these solutions present several limitations [8]:

- Each solution implements the BCI lifecycle differently, forcing researchers to use multiple platforms to perform their studies.

* Corresponding author.

E-mail addresses: slopez@um.es (S. López Bernal), juanantonio.martinezl@um.es (J.A. Martínez López), enriquetomas@um.es (E.T. Martínez Beltrán), mqp@um.es (M. Quiles Pérez), gregorio@um.es (G. Martínez Pérez), huertas@ifi.uzh.ch (A. Huertas Celdrán).

<https://doi.org/10.1016/j.neucom.2024.128027>

Received 19 December 2023; Received in revised form 19 May 2024; Accepted 7 June 2024

Available online 10 June 2024

0925-2312/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

For example, most software platforms implementing the acquisition phase are not generic, as they are developed to support communication with particular BCIs and manufacturers.

- All open-source BCI software platforms are desktop applications following a monolithic architecture despite the scalability problems such architecture can cause. Moreover, Most BCI solutions are not standalone, usually requiring complex installation and maintenance.
- These applications are conceived to be used offline, not considering the benefits of online solutions taking advantage of cloud architectures. In this sense, current open-source BCI software does not allow researchers to share experiments and results .

Aiming to improve the previous limitations, this work presents the following main contributions:

- The design and development of NeuronLab, a secure, multiplatform, standalone, multiparadigm, and web-based BCI framework that unifies all lifecycle phases to study EEG signals acquired by non-invasive BCIs. In particular, the proposed framework has been designed to be generic and compatible with a large variety of BCI devices based on EEG. Finally, the framework provides novel functionality compared to existing open-source platforms, taking advantage of backend capabilities such as remote and shared storage, allowing to share results between researchers.
- The deployment and evaluation of the framework in well-known experiments from the BCI domain, as (i) the detection of the P300 visual evoked potential, which can be intentionally elicited in the subject through the Oddball paradigm, consisting in the presentation of known and unknown stimuli to study the cerebral response [9], and (ii) the identification of movements in a driving scenario, such as moving the steering wheel, braking and accelerating.
- The analysis of the performance and scalability of NeuronLab from both application and server perspectives. It considers as metrics Central Processing Unit (CPU), Random Access Memory (RAM), disk consumption, and time required for each phase of the BCI lifecycle.

The remainder of the paper is structured as follows. Section 2 analyzes the related work of open-source BCI frameworks used in the literature. After that, Section 3 presents the design and development of NeuronLab, a BCI framework that overcomes the limitations found in the literature, while Section 4 validates the solution with two well-known BCI experiments. Subsequently, Section 5 details the results obtained during the validation of the framework based on performance metrics, also providing a comparison of NeuronLab with existing platforms. Finally, Section 6 presents conclusions and future work.

2. Related work

This section provides an analysis of the state of the art of both open-source software platforms and frameworks that allow the study of EEG signals acquired by non-invasive BCIs. The first category, *Commercial BCI applications*, groups the most used open-source commercial applications ready for final users, providing an environment for BCI research. The second one, *Research BCI frameworks*, analyzes emerging solutions developed for specific needs in BCI research, which aim to be an alternative to commercial software by providing novel functionality.

2.1. Commercial BCI applications

Software platforms dedicated to the BCI domain emerged in the early 2000s. These solutions eased the interaction with BCIs, allowing both researchers and practitioners to analyze EEG signals in a simpler way [10]. Thus, these applications are considered development toolkits, including the essential functionality required to perform the most

common BCI studies [11]. In particular, the leading software platforms used nowadays are subsequently presented.

BioSig is an Octave and MATLAB toolbox launched in 2003, which provides solutions for artifact preprocessing, quality control, feature extraction, data classification, and data visualization [12]. More in detail, the preprocessing module provides tools for data segmentation in signals, artifact detection, and artifact reduction to improve the quality of EEG signals. The classification module offers the following algorithms: Linear Discriminant Analysis (LDA), several Support Vector Machine (SVM) methods, Naive Bayes classifiers, perceptron learning, and Partial Least Regression (PLR) analysis. Finally, BioSig applies cross-validation procedures to avoid overfitting during data classification.

Letswave7 is a MATLAB toolbox that provides several functions for EEG signal preprocessing [13]. It implements preprocessing methods such as bandpass filter, notch filter, and Independent Component Analysis (ICA). Moreover, this platform provides enough functionality for EEG signal analysis in the time and frequency domains. It presents a modular architecture composed of four independent components. The manager module is the main interface in charge of EEG reading and writing management to files. In contrast, the batch module contains several preprocessing functions for data analysis and studying multiple datasets. Finally, the display module observes one or more datasets, whereas the figure module provides graphics generation.

OpenBCI GUI is a modular BCI application that only supports BCI hardware developed by the OpenBCI company [14]. It is composed of a central module, named multiplexor server, whose purpose is to connect new modules and establish communication between them. Additionally, OpenBCI GUI provides real-time EEG signal display for each BCI electrode, facilitating the initial calibration task. The EEG signals can also be preprocessed in real-time using bandpass and notch filters. Moreover, the EEG signals can be transmitted to other external processes via Lab Streaming Layer (LSL) protocol, User Datagram Protocol (UDP), serial communication, and Open Sound Control (OCS). Finally, this software does not include data classification capabilities, typically used together with other tools.

OpenViBE is a multiplatform application used for BCI design and testing [15]. It comprises a set of modules that can be integrated to develop fully functional BCIs. OpenViBE is divided into two applications. First, OpenViBE Designer allows creating and modifying experimental scenarios, displaying data, and selecting parameterizable functionalities. It provides the most common preprocessing and feature extraction techniques and includes several automatic learning algorithms such as SVM or Neural Networks (NNs). The second module is OpenViBE Acquisition Server, which establishes an interface for the acquisition of EEG signals.

EEGLab is a MATLAB and multiplatform toolbox for preprocessing EEG signals [10]. Its functionality includes data import, reporting, and visualization. EEGLab is divided into two layers. First, the top layer functions abstract the users and allow them to interact with the data through the GUI. Secondly, the lower layer allows users to customize the data preprocessing through command history and interactive functions. Concerning data preprocessing techniques, EEGLab implements a bandpass filter, notch filter, downsampling, and ICA. Finally, this tool does not support data classification based on artificial intelligence.

EEGFrame is a standalone software for visual inspection of the EEG signals, feature extraction, and storage of feature vectors developed in Java [16]. It allows to apply feature extraction methods to the signals stored in European Data Format (EDF) files and export them in Comma-Separated Values (CSV) files. It defines several categories of features: time domain, frequency domain, time–frequency domain, and non-linear features. This tool does not include either preprocessing or classification capabilities.

Table 1
State of the art of current BCI software.

Frameworks	OS			Implemented functionality											
	Windows	Linux	macOS	Acquisition	Genericity	Monitoring	Offline analysis	Preprocessing	Feature extraction	ML	DL	Standalone	Web-based	Security	
BioSig [12]	✓	✓	✓	✗	✗	✗	✓	✓	✓	✓	✓	✗	✗	✗	
Letswave7 [13]	✓	✓	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	
OpenBCI GUI [14]	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗	
OpenViBE [15]	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	
EEGLab [10]	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	
SecBrain [17]	✓	✓	✓	✓	✓	✗	✓	✓	✗	✓	✗	✗	✗	✓	
SAE+LSTM [18]	?	?	?	✗	✗	✗	✗	✓	✓	✗	✓	?	✗	✗	
EEGFrame [16]	✓	✓	✓	✗	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗	
Adaptive BCI framework [19]	?	?	?	✓	?	✗	✗	✓	✓	✓	✗	?	✗	✗	
NeuronLab	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	

✓ provided, ✗ not provided, ? not specified.

2.2. Research BCI frameworks

This section presents some research frameworks documented in the literature, aiming to surpass particular limitations of commercial BCI applications. Another reason to implement these frameworks is that they focus on specific BCI paradigms or solutions not covered by more general BCI platforms. However, these solutions tend to be domain-specific and suffer a lack of customization in their parameters, requiring changing the code to adapt to new requirements. Some examples of novel frameworks are presented as follows.

SecBrain is a Python framework that allows creating a secure and stable environment to implement scenarios based on the detection of the P300 utilizing the Oddball paradigm [17]. For that, the framework acquires the EEG signals from the subject using the LSL protocol. Then, after an exhaustive offline analysis of the EEG signals, the authors implemented a preprocessing phase consisting in a notch filter at 50 Hz, a bandpass in the range [3–17 Hz], ICA, and a downsampling from 250 Hz to 50 Hz. Once the signals are preprocessed, the framework uses five different classifiers to detect the P300 evoked potential.

SAE+LSTM is used for emotion recognition since it can help robots to interact with humans [18]. It obtains features from the EEG signals given as input using Stack Autoencoder (SAE) in a linear EEG mixing model to decompose the EEG signals. Subsequently, the signals are divided into short time frames, and a Power Spectral Density (PSD) in theta, alpha, beta, and gamma bands using Welch's method is applied. Finally, recognition models are trained by Long Short-Term Memory Recurrent Neural Networks (LSTM-RNN). Finally, this framework does not implement preprocessing techniques.

Adaptive BCI Framework is a system for online autocalibrating motor imagery BCIs [19]. The EEG signals are acquired and preprocessed with a bandpass filter in the range [0.5–100 Hz] and a notch filter at 50 Hz. The system is based on a two-class Graz BCI training paradigm, which recurrently selects only one out of six predefined logarithmic band power characteristics between [10–13 Hz] and [16–24 Hz] from Laplacian derivation [20]. Subsequently, the system retrains an LDA classification on all available data and updates the paradigm with the new model. The system does not require manual calibration since most parameters are selected online according to the current subject.

Grouping all software addressed in this section, [Table 1](#) shows a comparison between the literature and the proposed framework, taking into account certain aspects like their availability in the main desktop operative systems and the most relevant functionality they implement to determine the advantage of using the proposed solution. Finally, a more detailed comparison between existing applications and frameworks shown in this section and NeuronLab is provided in [Section 5.3](#).

3. NeuronLab framework

NeuronLab is a secure, multiplatform, and web-based framework that provides a working environment to researchers where every phase of the BCI lifecycle (acquisition, preprocessing, feature extraction, and classification) can be performed in a personalized way. Additionally, it implements sophisticated tools to study EEG signals, incorporating novel functionality that current open-source commercial applications and frameworks do not include. The source code of NeuronLab is publicly available in GitHub [21]. Based on the above, this section first presents the functionality offered by NeuronLab, followed by the analysis of the design and implementation of the framework.

3.1. NeuronLab functionality

The analysis performed in [Section 2](#) provided the current capabilities of the most common open-source software platforms and frameworks, highlighting the heterogeneity of the solutions and their limitations, thus generating an opportunity to create new software. In this sense, NeuronLab provides the following main functionality:

- **Users management:** The framework offers researchers to add new study subjects to the system, incorporating relevant information such as name, surname, and existing mental conditions.
- **Experiments management:** Researchers can create personalized BCI experiments according to their needs. Within the experiment, Neuronlab allows parameterizing all the required information to perform its execution, such as the BCI device to be used, the events expected to be received during data recording, and the subjects participating.
- **Experiments sharing:** An essential feature of this kind of application is to allow researchers to access the experiments created by other researchers. This way, NeuronLab offers the functionality to add new recordings to authorized researchers and even improve existing solutions with better strategies and results, obtaining more precise and complete experiments. It is worth mentioning that this functionality is only present nowadays in paid software.
- **Datasets management:** NeuronLab allows the interaction with the datasets generated during the experimentation, visually presenting characteristic information of the signal per channel. Thus, it allows researchers to perform offline analysis of the acquired data. If researchers require to use another BCI application, NeuronLab offers the functionality to download datasets in CSV format.
- **Unification of the BCI lifecycle:** The framework has been designed and implemented to completely cover all phases of the BCI cycle, not requiring external solutions for particular actions. Moreover, NeuronLab allows the user to parameterize the techniques used over the data concerning data preprocessing, feature extraction, and classification.

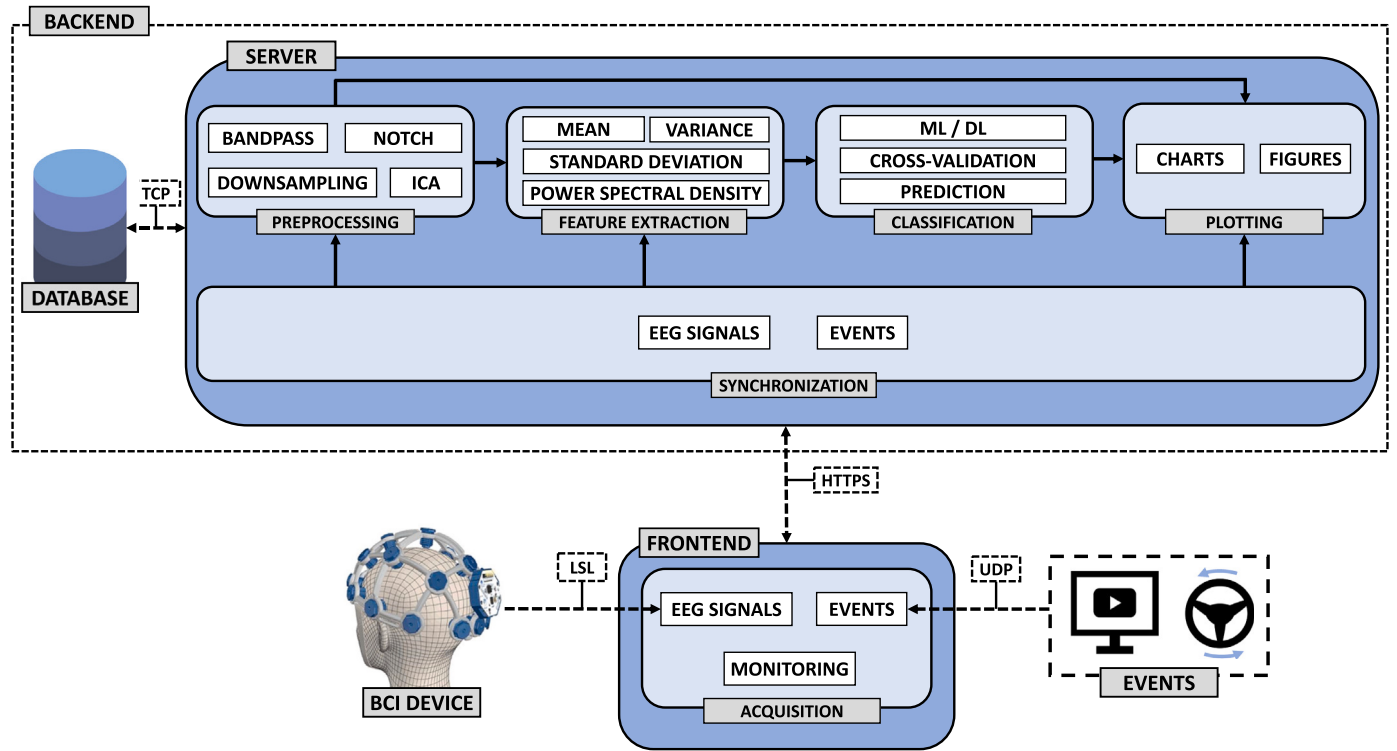


Fig. 1. High-level architecture of NeuronLab.

3.2. Overall architecture

The framework presents a web-based architecture, generating a clear separation between two layers: frontend and backend. Both layers include components or modules that handle specific system functionalities and provide dynamic communication with each other. A general solution diagram is depicted in Fig. 1, which shows the layers and components of the solution, the interaction between components, and the implemented communication protocols. The communication between components is unidirectional since the output from one module is given to the next as input. The main functionality of the components is summarized as follows:

- **Acquisition:** This component is deployed in the frontend and aims to acquire and monitor both subjects' EEG signals and events related to an experimental session.
- **Synchronization:** This module is part of the backend and focuses on synchronizing and storing the acquired data coming from the frontend.
- **Preprocessing:** It removes noise or artifacts of the acquired EEG signals to improve their quality.
- **Feature extraction:** It obtains characteristic information from the EEG signals, reducing redundant and discriminating data by identifying a set of features.
- **Classification:** It focuses on classifying the acquired EEG signals during one recording. For that, it trains and evaluates supervised ML and DL models.
- **Plotting:** It creates useful figures and charts related to the information gathered from the previous components.

The current design of NeuronLab is not oriented to online BCI paradigms, which means that once models are trained, the framework does not provide capabilities for evaluating samples in real time. Nevertheless, adapting it to cover an online approach would not substantially change the architecture presented in Fig. 1.

3.3. Frontend

The frontend layer corresponds to a desktop application implemented with the Electron.js framework [22]. This framework allows the creation of cross-platform applications (also called multiplatform applications) that run in any operating system, offering a performance as good as a native application. Electron.js is based on Node.js, which permits the development of software with web technologies like HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript (JS). Moreover, Electron.js allows to implement more powerful Graphical User Interfaces (GUIs) using libraries and frameworks such as React.js or Angular. In particular, React.js was the best option due to its simplicity and rendering optimizations to create Single Page Applications (SPAs). Based on the above, the frontend of NeuronLab includes a GUI implemented with React.js.

3.3.1. Acquisition component

The acquisition component included in the frontend can handle two different data streams. The first data stream corresponds to the EEG signals acquired by each electrode of the BCI and the timestamp. Specifically, NeuronLab allows the integration of diverse information flows corresponding to different biosignals. For this, it was necessary to use a JavaScript library for LSL communication. The sole alternative existing was an unofficial library that provided partial functionality of the protocol, not directly usable for the implemented framework [23]. Thus, NeuronLab extends this library, including additional functionality such as the correction of possible latency between the BCI device and the framework, available in [24]. Consequently, NeuronLab manages the communication with BCIs and the reception of the EEG signals via LSL. In this sense, NeuronLab can communicate with any EEG-based BCI that is compatible with LSL.

The second data stream corresponds to the acquisition of the events. Fig. 1 shows two categories of events corresponding to different experiments. The first event is related to the image visualization experiment, while the second corresponds to a left turn in a driving scenario. The framework represents them with a unique numeric number. The value

of each event is not arbitrary, and researchers must set it during the creation of the specific experiment in the framework. The system also manages the reception of event data via UDP since it is a simple and fast communication protocol, where each message includes the event and the timestamp. The messages are structured based on the JavaScript Object Notation (JSON) format, which is a lightweight representation in comparison to other data structures, such as Extensible Markup Language (XML) [25].

To support data acquisition, the GUI displays the evolution of the voltage over time of each BCI electrode when the acquisition process is running. Therefore, researchers can detect the electrodes incorrectly positioned since inconsistencies in the signals can be easily spotted. In the same way, it displays the events acquired and their associated timestamp. Additionally, the system saves all information acquired in temporal files when a certain threshold is reached to avoid using excessive RAM resources of the computer during the recording process. This writing process runs synchronously to prevent information losses. Finally, considering both data streams, this component must run on the client side to reduce communication latency and protect the information received for each stream from external attackers.

3.4. Backend

The backend layer is composed of a server and a database. The server is a RESTful Application Programming Interface (API) written in Python since it supports valuable libraries for data management, EEG signals processing, Machine Learning (ML) and Deep Learning (DL) model training, and data analysis. NeuronLab also employs MySQL as Database Management System (DBMS), enabling features like high stability, robustness, and maturity. This layer integrates two components, in charge of the synchronization, preprocessing, feature extraction, classification, and plotting processes, detailed in this section in depth.

3.4.1. Synchronization component

This component receives and synchronizes the EEG signals and the events acquired in one session recording to allow the framework to correlate both dimensions. More in detail, the synchronization procedure associates each event with the corresponding sample of the EEG signals, using the value of every timestamp provided by both EEG samples and the events. Hence, the correlation allows structuring the data in Epochs, representing equal-duration samples of the EEG signals associated with only one event. Finally, once the data are synchronized correctly, the server saves it in persistent storage using the CSV format, containing the voltage acquired from each electrode and the associated event value.

3.5. Preprocessing component

The preprocessing component removes noise or artifacts that may alter the EEG signals, using the MNE library to implement all the available preprocessing algorithms in the framework. Currently, NeuronLab supports the most common preprocessing techniques used in the literature, such as bandpass filter, which attenuates frequencies outside of a given range; notch filter, which rejects signals in a specific frequency; downsampling, to reduce the sampling rate of the signals; and ICA, to remove artifacts that interfere with EEG signals, such as blinking or breathing. This component always generates and stores a new CSV file that contains the preprocessed EEG signals to prevent overwriting the original data.

3.6. Feature extraction

The feature extraction module extracts characteristic information from the EEG signals, allowing the classification module to train later intelligent models able to predict information from those signals accurately. In particular, the framework supports two dimensions of features, the time domain, and the frequency domain. In the former, the

EEG signals are represented as a function of time, where the framework provides the mean, variance, and standard deviation of the initial data. In the case of the frequency domain, the EEG signals are defined based on their frequency spectrum by applying the Fourier series and transformation. The only algorithm NeuronLab supports for this domain is PSD in alpha, beta, delta, gamma, and theta frequency bands, using Welch's method provided by the MNE library. It is essential to highlight that the framework extracts only one vector per epoch contained in the dataset. Therefore, the total number of vectors is the same as the number of events.

Additionally, the preprocessing and feature extraction components can perform batch operations on a collection of datasets. However, while the preprocessing component allows applying several algorithms in a row, the latter is limited to performing only one action per dataset. Finally, the system generates a new CSV file that includes the new data for each input CSV selected by the researcher.

3.6.1. Classification

The classification component applies supervised Artificial Intelligence (AI) to generate trained models based on the EEG signals labeled with the events. The framework provides ML algorithms using the Scikit-learn library, supporting SVM, K-Nearest Neighbors (KNN), and Random Forest (RF) algorithms. For DL, the system uses Keras and Tensorflow libraries to create NNs composed of dense layers. In particular, users can train each model with data included in different datasets only if they belong to the same experiment and their features are equivalent. Moreover, although there are alternatives to Tensorflow (e.g., Pytorch), the authors only included this library due to its simplicity. Nevertheless, these alternatives could be easily included in the future.

In the same way, as in the preprocessing and feature extraction components, the GUI provides different options so that the researcher can choose the most suitable for the experiment. Focusing on NNs, for example, the user can select diverse parameters such as the optimizer, the loss function, or the learning rate. Moreover, for each layer to be added to the network, the user can determine its type (e.g., dense, or convolutional), the number of neurons per layer, or the activation function, among others. Fig. 2 presents an example of the GUI to configure an NN model.

After the trained model is created, the server applies cross-validation to check the accuracy and precision of the model, splitting the data into testing and training. The resulting model is exported to a .joblib file if the ML algorithm is used or .h5 if the DL approach is applied. Finally, the framework allows to use the models generated for the experiment to predict the events stored in other datasets only if the dataset belongs to the experiment used to train the model, the dataset has not been used for building the model, and it contains the same features of the model.

3.6.2. Plotting

The plotting component utilizes the datasets generated by the synchronization, preprocessing, and classification components as input and the models generated by the classification module. In terms of the first two components, it can provide charts containing the EEG signals and the events stored in the same dataset. In addition, the plotting component uses the MNE library to generate figures where the researcher can visualize the epochs of the EEG signals for each electrode, brain activity (i.e., topographic map) of an Epoch, and PSD charts. Regarding the classification component, when a DL algorithm is applied, the plotting component uses the trained model to plot two figures, showing the accuracy of the model and its loss, respectively.

Fig. 2. Example of the GUI to configure the parameters of NN models.

3.7. Security mechanisms

The implemented framework has been conceived following a security-by-design approach, preventing unauthorized data access, use, modification, or deletion and, thus, providing data integrity, confidentiality, and availability. In particular, all the communication between the frontend and the server uses the Hypertext Transfer Protocol Secure (HTTPS) protocol, which provides data confidentiality and integrity. This way, the requests and responses are not sent in plain text, avoiding a man-in-the-middle attack. Secondly, user passwords are hashed and stored in the database when a new user signs up for the application. Thus, if attackers get access to the database, they will not be able to access the original password used by the researcher. The algorithm employed to secure the passwords is bcrypt, which is used to hash and salt passwords. The output is no longer predictable, protecting it against rainbow table attacks when a salt (random string) is used.

Thirdly, to prevent an attacker from having limitless access to sending requests to the API, the server incorporates the OAuth2 protocol to handle the authorization to the framework. Once the researcher has successfully logged in, the server creates a JSON Web Token (JWT) stored in the desktop application until the researcher logs out. With that token, the server can identify if the researcher who sent the request is authorized in the framework. It is also necessary to protect the confidentiality of the subjects of the experiments since brain signals are considered medical-grade information with high criticality. Particularly, subjects' names are encrypted when a CSV is created and stored in the database, preventing attackers with access to the database from associating the dataset with the corresponding subject. This functionality is initiated from the frontend, which manages the encryption and decryption of this private information. The cipher key used for both tasks is typed by the researcher when logging in and stored in the memory of the program, making it difficult for attackers to access it.

Finally, regarding service and data availability, NeuronLab has been conceived to continue working if the communication link gets disrupted. In this situation, the front end will store the data acquired locally and send it when the link is restored. The backend will remain listening until the communication is operational.

4. NeuronLab validation

This section presents key aspects for the validation of NeuronLab over two experiments commonly performed in the BCI literature. Moreover, it introduces the specific details used in the framework for each experiment, the BCI used for the experimentation, and the hardware where the frontend and the backend are tested.

Table 2

Events captured during the recording of the experiments.

Experiment	Duration (s)	Events	Stimuli
P300 detection	900	900	Non-target: 720 Target: 180
Limb detection	4380	440	Brake: 110 Accelerate: 110 Turn right: 110 Turn left: 110

4.1. Experiments evaluated

The suitability of the framework has been validated through two widespread experiments in the BCI literature, which are subsequently described. The first experiment corresponds to the detection of P300 potentials elicited by presenting visual stimuli to the subject. Specifically, it uses the Oddball paradigm to trigger the generation of this evoked potential when an image known to the subject is displayed within a large set of unknown pictures [26]. In particular, the experiment presented a collection of 180 images, where 20% of them corresponded to an image previously presented to the subject and, thus, known, while the 80% remaining were unknown. Then, the experiment displayed the images randomly on the screen, one per second, ending when all the images were presented to the user. The data available for this experiment was recorded for 15 min. Finally, this experiment defines two events in the framework, a “1” value when an unfamiliar image is shown and a “2” value otherwise.

In contrast, the second experiment addresses the detection of limb movements performed by the subject in a simulated driving scenario. The specific movements considered were turning right and left, braking, and accelerating. Each action was conducted 110 times with a ten-second interval between them, generating a total of 73 min of data. According to these actions, the experiment received four events: “1” for braking, “2” for accelerating, “3” for turning right, and “4” for turning left. Finally, Table 2 presents details concerning the events captured for each experiment.

It is essential to highlight that although NeuronLab has been tested over these two scenarios, it can cover many BCI use cases due to the variability of the data processing, feature extraction, and classification techniques implemented. Moreover, since NeuronLab is an open-source project, it can be extended by including new functionalities, many of which are already available in the open-source libraries it uses for each phase of the BCI lifecycle.

4.2. BCI for neural data acquisition

The BCI device used to acquire of EEG signals in both validation experiments was the OpenBCI Cyton biosensing board, composed of eight channels or electrodes, with a sampling rate of 250 Hz [27]. The channels selected for the first experiment were Fp1, Fp2, C3, C4, P7, P8, O1, and O2, mainly covering the occipital region of the brain. In contrast, the movement detection experiment targeted the motor cortex, using for that channels FC5, FC1, FC2, FC6, CP5, CP1, CP2, and CP6. This second experiment also required additional materials. On the one hand, the driving simulation was deployed using the CARLA simulator [28] since it provides easy integration with external hardware. On the other hand, the KROM K-Wheel controller offered a realistic setup to interact with the simulator, providing a steering wheel and pedals to accelerate and brake.

4.3. Implementation of the BCI lifecycle

Focusing first on the acquisition phase of the BCI cycle, the P300 experiment lasts 20 min and, considering that the sampling rate of the BCI device is 250 Hz, the generated dataset was composed of $250 \text{ (Hz)} \times 20 \text{ (min)} \times 60 \text{ (s)} = 300,000$ EEG signal samples. In contrast, as the duration of the recording in the driving experiment was 73 min, the dataset contained $250 \text{ (Hz)} \times 73 \text{ (min)} \times 60 \text{ (s)} = 1,095,000$ EEG samples.

Moving to the preprocessing stage, the algorithm used in the P300 detection experiment is a bandpass filter with the Finite Impulse Response (FIR) in the frequency range [3–17 Hz]. Additionally, the framework applies the notch filter with a fourth-order Butterworth filter to attenuate the frequency at 50 Hz because of the noise caused by the electrical cabling in Europe. In contrast, in the case of the driving experiment, NeuronLab uses bandpass filtering with the FIR superposition-addition method using a zero phase in the range [2–45 Hz]. Besides, the EEG signals are notch filtered as in the previous scenario.

For feature extraction, the P300 detection experiment used the mean of the EEG values for each channel. In contrast, the movements detection experiment used PSD of all available bands (alpha, beta, delta, gamma, theta). Finally, the classification stage used both ML and DL algorithms. In this case, the same algorithms were used in both experiments. On the one hand, the framework used the RF algorithm with 100 trees, 80% as training data, and 20% as testing data. On the other hand, NeuronLab trained a NN model composed of three dense layers with 12 nodes in the first layer, eight in the second, and a final layer with one node for the P300 detection approach and four for detecting movements, corresponding to each of the movements to detect. In addition, the number of epochs is 100, and the data are split into 80%–20% as well.

4.4. Framework deployment

The frontend layer of NeuronLab was deployed on two computers with different characteristics, allowing a better understanding of the behavior of the desktop application. In particular, Computer I is a desktop composed of Windows 11, AMD Ryzen 5 5600X CPU with six cores and two threads per core, Nvidia RTX 3070 as GPU, and 32 GB of RAM. Computer II is a MacBook Pro 2021 using MacOS 13.2 with an M1 Pro chip composed of an 8-core CPU (six performance cores and two efficient cores), a 14-core GPU, and 16 GB of RAM. In this case study, only one frontend instance was deployed per computer used.

Moreover, two modes were considered to evaluate the performance of the GUI. The first mode studies the consumption of the application for ten minutes when it was running in the background, not executing specific tasks. Oppositely, in the second mode, the frontend executes the different stages of the BCI cycle. Furthermore, the backend layer was also deployed in Computer I. Particularly, the server uses

Uvicorn, an Asynchronous Server Gateway Interface (ASGI) implementation for Python, with one worker to present a simple implementation of the proof of concept. On the contrary, this validation process uses FastAPI to manage simultaneous requests, which helps provide concurrent access to the framework [29].

4.5. Performance metrics

Four performance metrics are used to validate the framework: (i) mean CPU consumption, (ii) mean RAM usage, (iii) required storage consumption in MB, and (iv) total time required to complete the task (including data transmission if applicable). The framework uses the psutil Python library to obtain CPU and RAM usage, acquiring and storing samples every 10 ms. Since CPU, RAM, and time oscillate during the execution of each task, their variability is subsequently presented in Section 5, which evaluates all four metrics using a different number of parallel users.

5. Results and discussion

This section details the results based on metrics used for the validation (see Section 4.5) of all components related to the BCI lifecycle that compose the architecture of NeuronLab (acquisition, synchronization, preprocessing, feature extraction, and classification) in the experiments described in Section 4. Finally, this section presents a discussion of the proposed solution concerning the existing BCI applications and frameworks reported in the literature.

5.1. Frontend results

Table 3 shows the performance of the frontend when executed in the background and for both experiments tested. As can be seen, when the application is in background mode, it has a reduced CPU and RAM usage in the two computers used. However, these values increase during data recording since it is a process requiring a higher computational cost. In addition, it is essential to remind that the frontend, during the recording, stores both the EEG signal and the captured events in temporary files each time it gathers 30,000 EEG samples regardless of the experiment being executed, thus optimizing the use of the RAM. Thus, as the recording of the first experiment was 15 min, NeuronLab generated a total of eight temporary files, where the first seven required 4.82 MB and the last one 2.08 MB. In contrast, as the recording time of the second experiment was 73 min, the framework created 37 temporary files, the first 36 being 4.82 MB and the last 2.55 MB. Finally, it is important to mention that the time required for each experiment is directly related to the duration of the experiment itself.

5.2. Backend results

The server resource consumption was analyzed when one, five, ten, 20, and 50 users simultaneously employed each component related to the BCI lifecycle. This way, this section presents a scalability study when NeuronLab performs parallel tasks. First, it is important to mention that the backend, at rest without any user, requires up to 1.38% of CPU and 351 MB of RAM. Moreover, Table 4 highlights the results for these different numbers of simultaneous users, indicating the minimum and maximum values observed for each metric studied. This table highlights that the maximum consumption in all four metrics tends to increase when the number of users augments. It is essential to indicate that the storage linearly increases with the number of subjects, as each user utilizes the same dataset per experiment.

Focusing first on the synchronization and preprocessing phases, Table 4 indicates that the detection of limb movements requires more resources, as the datasets contain a higher recording duration than the P300 detection experiment and, thus, more samples to process. Moving

Table 3

Comparison of frontend resource consumption for background execution and data recording in both experiments.

Computer	Background		Recording (P300 detection)				Recording (Limb movements detection)			
	CPU (%)	RAM (MB)	CPU (%)	RAM (MB)	Disk (MB)	Time (s)	CPU (%)	RAM (MB)	Disk (MB)	Time (s)
I	0.0–1.19	264.84–287.15	0.0–14.33	353.97–754.09	35.82	900	0.0–15.53	405.37–582.39	176.07	4380
II	0.0–12.71	355.2–389.98	0.0–22.21	217.48–550.77	35.82	900	0.0–29.81	234.11–746.5	176.07	4380

to the extraction of features, detecting limb movements requires more CPU and RAM due to presenting a larger dataset. However, the disk usage and the time required are lower because the number of events acquired in this experiment is inferior. Finally, the metrics results for both classification alternatives indicate the resources required to train the models. It is important to note that the disk consumption corresponds to both the storage of the models and the charts representing the accuracy and loss of each model.

Furthermore, this section presents detailed figures analyzing the CPU, RAM, and time evolution for each of the concurrent users tested and each phase of the BCI lifecycle. First, Fig. 3 presents the mean CPU required for completing each phase, highlighting that the mean consumption for all phases in both experiments is lower than 10%. It is also relevant to highlight that the CPU presents a high variability due to the allocation of resources performed internally by the operating system. In contrast, Fig. 4 highlights the mean RAM usage required by both experiments. Interestingly, in the P300 experiment, there are not many differences between the different stages. In the detection of limb movements, there is a clear separation between the stages, the synchronization and preprocessing phases being the most consuming. In both cases, the RAM usage is close to a linear trend when increasing the number of parallel users. Finally, Fig. 5 presents the mean time required to complete each phase. This figure indicates that the synchronization phase is the one requiring most of the time due to its dependence on the input data. As for the RAM metric, the time increases proportionally when augmenting the number of concurrent users using NeuronLab.

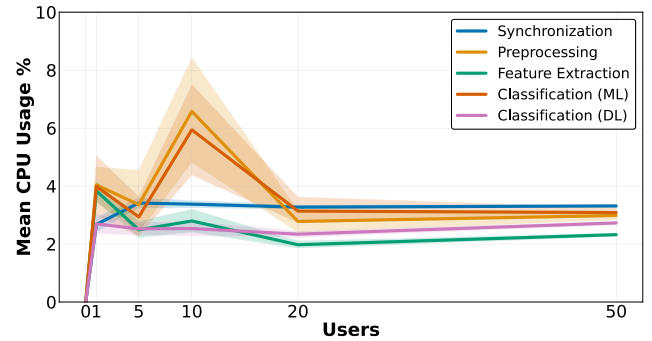
It is also relevant to indicate the performance of the models trained for each experiment. As can be seen in Table 5, the detection of P300 events obtains acceptable results, between 75%–82% F1-score, depending on the algorithm used. Using a neural network decreases the performance due to the limited data available. In contrast, the detection of limb movements decreases performance, ranging between 55%–61%. This performance reduction compared to P300 is coherent as this second experiment focuses on a multi-class process that has to identify between four possible classes. All these results are aligned with those obtained in the literature [30,31]. Nevertheless, this paper does not intend to comprehensively study or improve the performance of the results existing in the literature, only focusing on the performance of NeuronLab as a framework.

5.3. Discussion

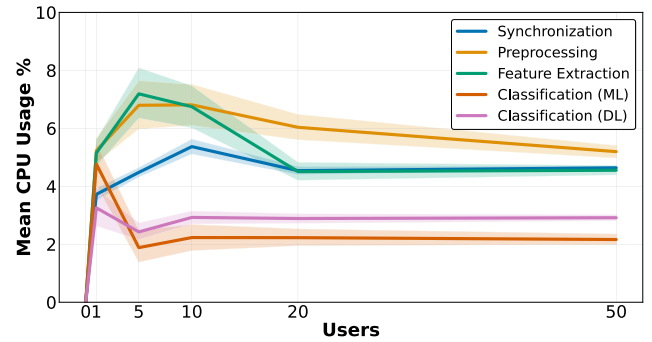
After validating the proposed solution, this section compares NeuronLab with other commercial BCI applications and research BCI frameworks previously shown in Table 1. This discussion focuses on the implemented functionalities these solutions provide since they cannot be compared according to the experiments performed in this work.

Focusing first on the software architectures used by existing solutions studied, they all follow a monolithic approach, where the whole software runs on a desktop computer. In contrast, NeuronLab offers both a desktop client to interact with the BCI and personalize the experiments and a server capable of performing most of the computing capabilities of the framework. Thus, NeuronLab proposes using a web-based architecture, surpassing the limitations of current solutions in terms of scalability and performance.

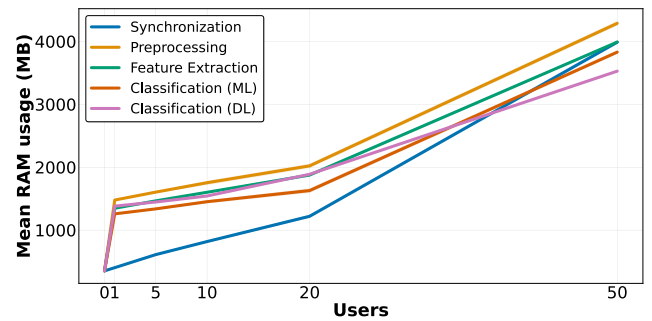
Another problem of current BCI solutions is that they do not tend to be standalone and are difficult to install when they require external applications. This limitation is solved by NeuronLab, as this framework has been designed not to depend on other external tools. In this context,



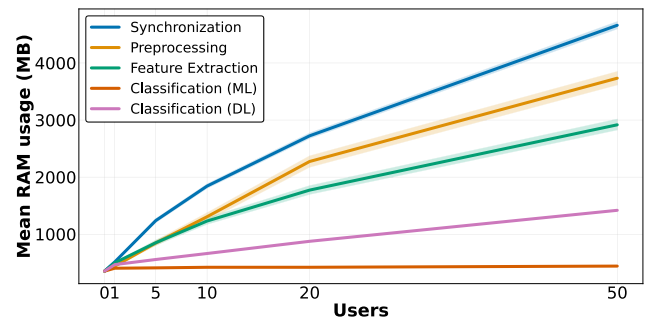
(a) P300 detection experiment.



(b) Limb movements detection experiment.

Fig. 3. CPU usage for each BCI cycle phase.

(a) P300 detection experiment.



(b) Limb movements detection experiment.

Fig. 4. RAM memory usage for each BCI cycle phase.

Table 4

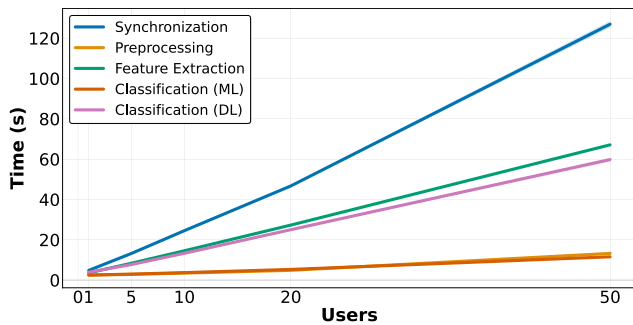
Results of metrics for all server components related to the BCI lifecycle.

	Number of users	P300 detection				Limb movements detection			
		CPU (%)	RAM (MB)	Disk (MB)	Time (s)	CPU (%)	RAM (MB)	Disk (MB)	Time (s)
Synchronization	1	1.0–6.98	351.72–463.46	38.9	4.65–4.98	1.0–9.38	351.97–889.4	140.04	12.11–13.0
	5	1.0–9.38	394.71–933.5	194.5	13.02–13.4	1.0–28.48	403.86–3138.95	702.2	48.07–49.69
	10	1.0–15.35	508.0–1533.73	389	23.78–25.34	1.0–45.98	589.42–4988.73	1400.4	90.72–97.12
	20	1.0–16.54	645.14–2766.78	778	45.84–47.45	0.26–49.53	846.66–9782.47	2800.8	183.05–187.02
	50	1.0–14.15	2812.88–6782.01	1945	124.19–129.35	0.0–65.62	1152.92–18651.84	7002	456.59–470.96
Preprocessing	1	2.21–5.73	1445.22–1520.61	39.5	2.51	1.01–9.38	356.56–526.1	161	3.59–3.8
	5	1.0–9.38	1547.87–1657.61	197.5	2.82–2.84	1.0–22.52	360.28–1195.11	815	4.55–4.72
	10	1.0–20.12	1542.08–1890.21	395	3.47–3.5	1.0–21.98	365.26–2155.33	1630	6.84–6.95
	20	1.0–14.15	1563.24–2245.18	790	4.79–4.9	1.0–26.42	376.34–3616.43	3260	11.73–12.07
	50	1.0–9.28	3148.45–4814.23	1975	13.16–13.38	1.0–24.9	394.03–6836.34	8050	26.82–27.89
Features extraction	1	1.0–6.98	1325.88–1380.42	0.167	3.48–3.6	1.0–8.17	433.52–621.05	0.135	3.46–3.48
	5	1.0–11.76	1383.17–1563.36	0.835	8.37–8.37	1.0–17.58	391.02–1501.55	0.675	4.23–4.26
	10	1.0–22.22	1316.01–1851.94	1.67	14.56–14.67	1.0–22.31	396.01–2545.81	1.35	6.08–6.3
	20	1.0–11.76	1323.66–2283.7	3.34	27.04–27.81	1.0–27.29	392.6–3487.54	2.7	10.42–10.75
	50	1.0–8.17	3183.49–4631.56	8.35	66.47–67.56	1.0–14.15	399.2–6391.09	6.75	23.89–24.32
Classification (ML)	1	1.0–9.38	1241.91–1272.81	1.63	2.5–2.51	3.37–5.79	402.2–403.45	0.045	2.27–2.29
	5	1.0–8.22	1249.45–1367.88	8.15	2.93–3.07	1.0–4.59	407.13–410.31	0.225	2.61–2.71
	10	1.0–24.68	1111.54–1556.03	16.3	3.7–3.82	1.0–5.79	412.14–422.66	0.45	3.14–3.16
	20	1.0–14.15	1122.66–1866.75	32.6	5.23–5.33	1.0–5.79	391.23–431.07	0.9	4.01–4.13
	50	1.0–9.38	3209.8–4345.64	81.5	11.43–11.66	1.0–5.79	417.73–444.63	2.25	6.95–6.96
Classification (DL)	1	1.0–6.98	1365.37–1391.12	0.194	3.8–4.02	1.0–6.98	453.57–477.73	0.02	2.81–3.26
	5	0.74–11.65	1369.73–1490.66	0.97	7.6–7.82	1.0–11.65	480.94–591.55	0.1	4.89–5.12
	10	1.0–16.44	1372.89–1746.83	1.94	13.03–13.46	0.86–10.57	590.45–703.54	0.2	7.5–7.93
	20	1.0–12.96	1428.32–2257.04	3.88	24.77–25.42	1.0–14.15	703.09–979.17	0.4	13.36–13.79
	50	1.0–9.38	2914.54–4260.69	9.7	57.78–60.83	1.0–14.03	978.62–1648.14	1	30.84–31.6

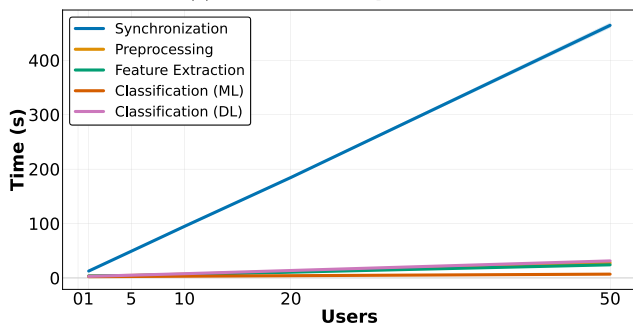
Table 5

Performance of the models trained to detect P300 and limb movements, offering percentage results.

P300 detection								Limb movement detection							
Random forest				Neural network				Random forest				Neural network			
accuracy	precision	recall	F1-score	accuracy	precision	recall	F1-score	accuracy	precision	recall	F1-score	accuracy	precision	recall	F1-score
86.40	85.28	77.93	82.46	76.17	71.08	81.48	75.89	65.61	69.97	54.60	61.34	59.90	62.47	49.58	55.28



(a) P300 detection experiment.



(b) Limb movements detection experiment.

Fig. 5. Required time for each BCI cycle phase.

although BCI platforms and frameworks are compatible with the most common operating systems, they have extensibility limitations. For example, all BCI software studied that can run in MacOS was compiled for Intel architecture, providing a lower performance on new Mac computers with chip ARM because of the need for Rosetta 2. In contrast, the NeuronLab desktop application is the only one optimized for such chips, as it uses Electron.js [32].

Focusing on the BCI lifecycle, the need for software to acquire EEG signals and stimuli is fundamental. However, BioSig, NPXLab Suite, Letswave7, SAE+LSTM, and EEGFrame do not provide this functionality. In this way, researchers would have to use at least two applications, one for acquiring and one for performing the rest of the phases. In addition to this limitation, OpenBCI GUI does not allow the acquisition of EEG signals if the OpenBCI company does not provide the headset.

Furthermore, the visualization of the EEG signals and the stimuli in real time is essential for researchers since they can understand the behavior of the signals when users face the stimuli, detecting the misplacement of electrodes. Moreover, the processing and analysis of the signals are helpful for researchers, although novel frameworks such as SAE+LSTM and Adaptive BCI Framework do not provide these capabilities. In addition, researchers typically need to extract features from the EEG signals to train a classifier using ML or DL algorithms. Nevertheless, Letswave7, OpenBCI GUI, and EEGLab do not provide these functionalities. In addition, most tools provide ML algorithms but not DL algorithms. According to these limitations, NeuronLab unifies the preprocessing, feature extraction, and ML/DL classification on the same platform.

Finally, most of the existing solutions do not consider or implement security mechanisms despite the critical information contained in the EEG signals. Thus, these platforms should progress to offer the securitization of such sensitive assets. NeuronLab, unlike these

solutions, implements some mechanisms to provide data confidentiality and integrity.

6. Conclusion

This work proposes the design and implementation of NeuronLab, a secure, open-source, and web-based framework that implements all phases of the BCI lifecycle (acquisition, preprocessing, feature extraction, and classification) in a unified way. For the first phase, it can acquire EEG signals from non-invasive BCI devices, being easily extended to support other biosensor devices. The solution then provides signal preprocessing, feature extraction capabilities, and ML and DL classification techniques. Furthermore, NeuronLab offers functionality not found in other open-source software solutions, such as experiment sharing between different researchers, which is intended to achieve more complete experiment results, subject management, and storing files on the cloud.

NeuronLab has been tested over two realistic and common experiments in the BCI literature. The first experiment detected P300 evoked potentials within EEG signals generated using the Oddball paradigm, where subjects visualized familiar and unfamiliar images. The second experiment focused on limb movement detection in a simulated driving scenario, such as turning right and left, braking, and accelerating. This scenario was composed of a driving simulator, steering wheels, and pedals. Moreover, both experiments utilized a BCI headset to acquire subjects' EEG signals. Finally, this validation used metrics of system resources such as CPU, RAM, and disk consumption, as well as the time required to complete each phase of the BCI cycle to determine the performance of frontend and backend software components.

Future work could focus on implementing an architecture based on microservices to increment the scalability of the backend. In addition, it is intended to add new preprocessing, feature extraction, and classification techniques to enrich the capabilities of the framework. Finally, this work proposes adding and evaluating new biosensors, such as eye-tracking glasses or devices measuring glucose, to increase the extensibility of the framework. In this manner, the framework could be a better solution for researchers, allowing them to implement other experiments outside the BCI domain.

Funding

This work has been partially supported by (a) 21628/FPI/21 and 21629/FPI/21 Fundación Séneca, cofunded by Bit & Brain Technologies S.L. Región de Murcia (Spain), (b) the strategic project CDL-TALENTUM from the Spanish National Institute of Cybersecurity (INCIBE) and by the Recovery, Transformation and Resilience Plan, Next Generation EU, (c) the Swiss Federal Office for Defense Procurement (armasuisse) with the CyberTracer (CYD-C-2020003) project, and (d) the University of Zürich UZH.

CRediT authorship contribution statement

Sergio López Bernal: Writing – original draft, Methodology, Data curation. **Juan Antonio Martínez López:** Writing – original draft, Software, Methodology, Data curation. **Enrique Tomás Martínez Beltrán:** Writing – original draft, Software, Methodology. **Mario Quiles Pérez:** Writing – original draft, Methodology, Data curation. **Gregorio Martínez Pérez:** Writing – review & editing, Methodology, Conceptualization. **Alberto Huertas Celdrán:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Enrique Tomas Martinez Beltran reports financial support was provided by Fundación Séneca. Mario Quiles Perez reports financial support was provided by Fundación Séneca. Alberto Huertas Celdran reports financial support was provided by Swiss Federal Office for Defense Procurement (armasuisse). Alberto Huertas Celdran reports financial support was provided by University of Zurich. Sergio Lopez Bernal reports financial support was provided by Spanish National Institute of Cybersecurity (INCIBE). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code is available at GitHub and adequately cited within the manuscript.

References

- [1] M.B. de la Mora Mújica, J.A. Río Portilla, Biosensores, 2018, URL: <https://conogasi.org/articulos/biosensores/>.
- [2] A. Kawamura, T. Miyata, 4.2 - Biosensors, in: M. Ebara (Ed.), Biomaterials Nanoarchitectonics, William Andrew Publishing, 2016, pp. 157–176, <http://dx.doi.org/10.1016/B978-0-323-37127-8.00010-8>.
- [3] M. van Gerven, J. Farquhar, R. Schaefer, R. Vlek, J. Geuze, A. Nijholt, N. Ramsey, P. Haselager, L. Vuurpijl, S. Gielen, P. Desain, The brain-computer interface cycle, J. Neural Eng. 6 (2009) 041001, <http://dx.doi.org/10.1088/1741-2560/6/4/041001>.
- [4] E. Khabarova, N. Denisova, A. Dmitriev, K. Slavin, L. Verhagen Metman, Deep brain stimulation of the subthalamic nucleus in patients with parkinson disease with prior pallidotomy or thalamotomy, Brain Sci. 8 (4) (2018) 66, <http://dx.doi.org/10.3390/brainsci8040066>.
- [5] R. İnce, S.S. Adanır, F. Sevmez, The inventor of electroencephalography (EEG): Hans Berger (1873–1941), Childs Nerv. Syst. 37 (9) (2021) 2723–2724.
- [6] M. Quiles Pérez, E. Martínez Beltrán, S. López Bernal, A. Huertas, G. Martínez Pérez, Breaching subjects' thoughts privacy: A study with visual stimuli and brain-computer interfaces, J. Healthc. Eng. 2021 (2021) 1–12, <http://dx.doi.org/10.1155/2021/5517637>.
- [7] S. López Bernal, A. Huertas Celdrán, G. Martínez Pérez, M.T. Barros, S. Balasubramaniam, Security in brain-computer interfaces: State-of-the-art, opportunities, and future challenges, ACM Comput. Surv. 54 (1) (2021) <http://dx.doi.org/10.1145/3427376>.
- [8] P. Stegman, C.S. Crawford, M. Andujar, A. Nijholt, J.E. Gilbert, Brain-computer interface software: A review and discussion, IEEE Trans. Hum.-Mach. Syst. 50 (2) (2020) 101–115, <http://dx.doi.org/10.1109/THMS.2020.2968411>.
- [9] M. Lindín, M. Zurrón, F. Díaz, Changes in P300 amplitude during an active standard auditory oddball task, Biol. Psychol. 66 (2) (2004) 153–167, <http://dx.doi.org/10.1016/j.biopsycho.2003.10.007>.
- [10] A. Delorme, S. Makeig, EEGLAB: an open source toolbox for analysis of single-trial EEG dynamics including independent component analysis, J. Neurosci. Methods 134 (1) (2004) 9–21, doi: <https://doi.org/10.1016/j.jneumeth.2003.10.009>.
- [11] J. Lindgren, A. Lecuyer, OpenViBE and other BCI software platforms, in: Brain-Computer Interfaces 2, John Wiley & Sons, Ltd, 2016, pp. 179–198, <http://dx.doi.org/10.1002/9781119332428.ch10>.
- [12] C. Vidaurre, T. Sander, A. Schlögl, BioSig: The free and open source software library for biomedical signal processing, Comput. Intell. Neurosci. 2011 (2011) 935364, <http://dx.doi.org/10.1155/2011/935364>.
- [13] Letswave, Letswave7, 2018, URL: <https://letswave.cn>.
- [14] OpenBCI, s.f., The OpenBCI GUI, URL: <https://docs.openbci.com/Software/OpenBCISoftware/GUIDocs/>.
- [15] Y. Renard, F. Lotte, G. Gibert, M. Congedo, E. Maby, V. Delannoy, O. Bertrand, A. Lecuyer, OpenViBE: An open-source software platform to design, test and use brain-computer interfaces in real and virtual environments, Presence Teleoper. Virtual Environ. / Presence Teleoper. Virtual Environ. 19 (2010) <http://dx.doi.org/10.1162/pres.19.1.35>.
- [16] A. Jović, L. Suć, N. Bogunović, Feature extraction from electroencephalographic records using EEGFrame framework, in: 2013 36th International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO, IEEE, 2013, pp. 965–970.

- [17] E. Martínez Beltrán, M. Quiles, S. López Bernal, A. Huertas Celdrán, G. Martínez Pérez, SecBrain: A framework to detect cyberattacks revealing sensitive data in brain-computer interfaces, in: *Advances in Malware and Data-Driven Network Security*, IGI Global, 2021, p. 23, <http://dx.doi.org/10.4018/978-1-7998-7789-9.ch010>.
- [18] X. Xing, Z. Li, T. Xu, L. Shu, B. Hu, X. Xu, SAE+LSTM: A new framework for emotion recognition from multi-channel EEG, *Front. Neurobot.* 13 (2019) <http://dx.doi.org/10.3389/fnbot.2019.00037>.
- [19] J. Faller, C. Vidaurre, T. Solis-Escalante, C. Neuper, R. Scherer, Autocalibration and recurrent adaptation: Towards a plug and play online ERD-BCI, *IEEE Trans. Neural Syst. Rehabil. Eng.* 20 (3) (2012) 313–319, <http://dx.doi.org/10.1109/TNSRE.2012.2189584>.
- [20] C. Guger, A. Schlögl, C. Neuper, D. Walterspacher, T. Strein, G. Pfurtscheller, Rapid prototyping of an EEG-based brain-computer interface (BCI), *IEEE Trans. Neural Syst. Rehabil. Eng.* 9 (1) (2001) 49–58, <http://dx.doi.org/10.1109/7333.918276>.
- [21] J.A. Martínez López, NeuronLab GitHub repository, 2023, URL: <https://github.com/JuananMtez/NeuronLab>.
- [22] F. OpenJS, Electron | build cross-platform desktop apps with JavaScript, HTML, and CSS, 2022, URL: <https://www.electronjs.org>.
- [23] U. Shaked, Urish/node-lsl, 2017, URL: <https://github.com/urish/node-lsl>.
- [24] J.A. Martínez López, Implementation of LSL for JavaScript, 2023, URL: <https://github.com/JuananMtez/lsl.js>.
- [25] S. Zunke, V. D'Souza, Json vs xml: A comparative performance analysis of data exchange formats, *IJCSN Int. J. Comput. Sci. Netw.* 3 (4) (2014) 257–261.
- [26] E.T. Martínez Beltrán, M. Quiles Pérez, S. López Bernal, A. Huertas Celdrán, G. Martínez Pérez, Noise-based cyberattacks generating fake P300 waves in brain-computer interfaces, *Cluster Comput.* 25 (1) (2022) 33–48, <http://dx.doi.org/10.1007/s10586-021-03326-z>.
- [27] OpenBCI, OPENBCI CYTON biosensing board (8-Channels), 2022, URL: <https://shop.openbci.com/products/cyton-biosensing-board-8-channel>.
- [28] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: S. Levine, V. Vanhoucke, K. Goldberg (Eds.), *Proceedings of the 1st Annual Conference on Robot Learning*, in: *Proceedings of Machine Learning Research*, Vol. 78, PMLR, 2017, pp. 1–16.
- [29] S. Ramírez Montaña, FastAPI, 2022, URL: <https://fastapi.tiangolo.com>.
- [30] L.G. Hernández-Rojas, O.M. Montoya, J.M. Antelis, Anticipatory detection of self-paced rehabilitative movements in the same upper limb from EEG signals, *IEEE Access* 8 (2020) 119728–119743, <http://dx.doi.org/10.1109/ACCESS.2020.3005600>.
- [31] K. Won, M. Kwon, M. Ahn, S.C. Jun, EEG dataset for RSVP and P300 speller brain-computer interfaces, *Sci. Data* 9 (1) (2022) 388, <http://dx.doi.org/10.1038/s41597-022-01509-w>.
- [32] S. Attard, Apple silicon support, 2020, URL: <https://www.electronjs.org/blog/apple-silicon>.



Sergio López Bernal is a postdoctoral researcher at the University of Murcia, obtaining his Ph.D. from this university. He holds a M.Sc. in Computer Engineering and a B.Sc. in Computer Engineering from the University of Murcia, and a M.Sc. in Architecture and Engineering for the Internet of Things from IMT Atlantique, France. His research interests include cybersecurity, brain-computer interfaces, artificial intelligence, and data analysis.



Juan Antonio Martínez López is an M.Sc. degree student in New Technologies in the specialization of Technologies of the Software at the University of Murcia. Currently, he is working on the master thesis project which consists of a web software application for detection malware in IoT devices using Machine Learning and Deep Learning algorithms.



Enrique Tomás Martínez Beltrán is working toward a Ph.D. in Computer Science at the University of Murcia, Spain. He obtained a B.Sc. degree in Information and Communication Technologies and an M.Sc. degree in New Technologies, specializing in information security, networks, and telematics. His research interests include cybersecurity, Federated Learning, Brain-Computer Interfaces, IoT, and Artificial Intelligence applied to different fields using Machine Learning and Deep Learning techniques.



Mario Quiles Pérez is a Ph.D. student at the University of Murcia. He obtained his B.Sc. Degree in the specialization of networks, and later a M.Sc. in new technologies. For the realization of his thesis the Séneca foundation together with the company Bitbrain financed his study on how different biosignals can help in neuromarketing scenarios.



Gregorio Martínez Pérez received a Ph.D. degree in Computer Science at the University of Murcia, where he is Full Professor since 2014. His scientific activity is mainly devoted to cybersecurity and data science.



Alberto Huertas Celdrán is currently a Postdoctoral Researcher Associated with the Communication Systems Group at the University of Zurich, Switzerland. He received the M.Sc. and Ph.D. degrees in computer science from the University of Murcia, Spain.